

Enrichment Using Conceptual Spaces via Comments

A Framework for Multi-Stage Document Processing

Your Name

Your Institution

August 12, 2025

The Fundamental Idea

Two-Phase Enrichment Process

- 1 **Parsing Phase:** Multiple interpreters process documents and store results as **structured comments**
- 2 **Enrichment Phase:** Subsequent parsers use stored comments to **adapt their interpretation state**

Key Innovation

Conceptual spaces provide the **unifying framework** that connects:

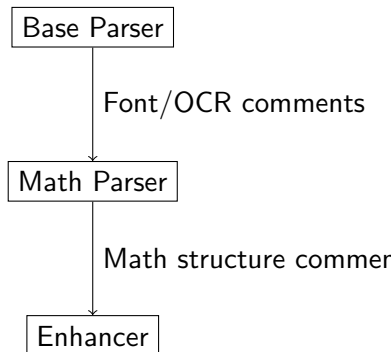
- Document structure analysis
- Semantic interpretation
- Context-aware processing

Placeholder for Expansion

[Insert conceptual diagram showing comment propagation between interpreters]

Parser Ecosystem Architecture

- **Stage 1:** Base parser (e.g., PDF/text extraction)
- **Stage 2:** Context-aware parser (e.g., LaTeX math interpreter)
- **Stage 3:** Enhancement parser (e.g., line continuation optimizer)



Placeholder for Expansion

[Insert specific LaTeX code example showing comment propagation]

LaTeX Math Processing Pipeline

Problem Statement

Traditional OCR and parsing fail to account for:

- Package-dependent math environments (amsmath, mathtools)
- Line continuation requirements
- Font-based symbol disambiguation

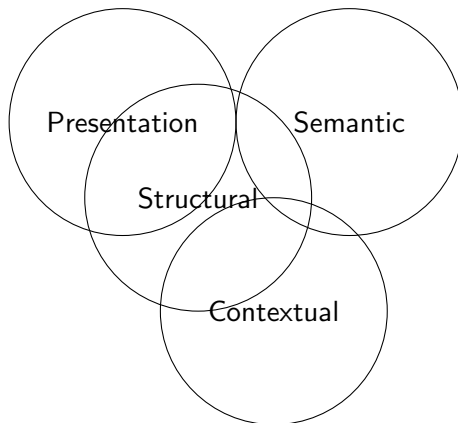
Our Solution

- 1 Merge PDF font metrics with MathPix OCR output
- 2 Store package context as comments in intermediate representation
- 3 Use comments to select appropriate math parser configuration

Placeholder for Expansion

[Insert concrete example with before/after line continuation]

Unifying Document Processing Dimensions



Comment Propagation Mechanism

- Each circle represents a dimension in the conceptual space
- Comments serve as **coordinates** in this multi-dimensional space
- Parsers navigate the space using comment-based guidance

Research and Development Pathways

Immediate Next Steps

- Implement prototype comment-aware LaTeX parser
- Develop standard comment schema for document processing

Long-Term Vision

- Create open standard for cross-parser communication
- Integrate with existing document processing ecosystems
- Extend to non-TeX formats (Markdown, XML, etc.)

Placeholder for Expansion

[Insert roadmap timeline with milestones]

Questions?

Contact: your.email@example.com

GitHub: github.com/yourusername/conceptual-space-comments

Constructor: BibliographCSP(bibkey)

Core Initialization Function

The constructor establishes the foundational structure for the conceptual space framework by initializing the CSP (Constraint Satisfaction Problem) base with a specific bibliography key. This creates the initial "geometric structure that represents a number of quality dimensions" for document processing.

- **Parameters:** bibkey (reference identifier for bibliography context)
- **Inheritance:** Extends CSP class to implement conceptual space principles
- **Role:** Sets up the dimensional framework where subsequent processing occurs

This initialization represents the first step in creating "a collection of quality dimensions divided into domains" specific to bibliography processing.

Multi-Stage Processing Function

This function implements the core enrichment process by parsing full text through multiple interpreters and storing intermediate results as structured comments. It embodies the "meeting of minds" concept where different processing stages communicate through shared conceptual representations.

- **Parameters:** `text` (document content to process)
- **Workflow:**
 - 1 Parse BibTeX entries from text
 - 2 Convert each entry to Tiddler objects
 - 3 Store results in a Map for downstream processing
- **Output:** Map of processed bibliography entries

This function realizes the principle that "comments are used to remind yourself and to inform others about the details of the code", but extends it to document processing where comments inform subsequent interpreters.

BibTeX Parsing: `parseBibTeX(text)` & `parseFields(fieldsText)`

Hierarchical Parsing Functions

These complementary functions implement the dimensional decomposition of document elements into the conceptual space framework. They transform raw text into structured representations that can be used as "theoretical entities that can be used to explain and predict various empirical phenomena" .

- **Parameters:**

- `parseBibTeX`: text (BibTeX formatted content)
- **Parameters:** `fieldsText` (individual entry fields)

- **Mechanism:**

- Uses regex to extract entries and fields
- Creates structured objects representing document elements
- Preserves hierarchical relationships between elements

- **Output:** Parsed entries/fields ready for conceptual mapping

These functions enable the transformation of "comments (Comment

Representation Conversion: bibEntryToTiddler(entry)

Conceptual Space Mapping Function

This function implements the critical transformation from parsed document elements to their representation within the conceptual space. It creates "formal conceptual spaces [that] can be exploited for the matching of concepts between parties of a communication process" .

- **Parameters:** entry (parsed BibTeX entry)
- **Transformation:**
 - Converts entry fields to Tiddler properties
 - Adds metadata for conceptual space positioning
 - Creates title using bibkey namespace
- **Output:** Tiddler object with conceptual space coordinates

The function exemplifies how "these mental spaces cover all domains of thought (and thoughtful action), from chemistry to choreography, from pottery to poetry" , by creating domain-specific representations within the shared conceptual framework.

Core Infrastructure: CSP Base Class

Foundation of the Conceptual Space Framework

The CSP base class implements the fundamental architecture for the "enrichment using conceptual spaces via comments" paradigm. It provides the essential structure for storing and managing document elements within a conceptual space framework.

- **Constructor:** `CSP(bibkey, tags = "")` initializes the conceptual space with reference identifiers
- **Data Management:** `add(id, content, extraFields)` stores elements as coordinates in conceptual space
- **Integration:** `appendTiddlers(tiddlers)` combines processed elements into the final document representation

This implementation realizes the principle that "comments are used to remind yourself and to inform others about the details of the code", but extends it to document processing where comments inform subsequent interpreters.

Multi-Stage Text Enhancement Function

This function implements a sophisticated pipeline for document cleaning and enrichment that demonstrates the core concept of "enrichment using conceptual spaces via comments." It processes text through multiple conceptual filters while preserving semantic integrity.

- **Workflow:**

- 1 Corrects line feed artifacts
- 2 Handles mathematical expressions
- 3 Identifies and preserves technical terms
- 4 Applies spellchecking with domain-specific dictionaries
- 5 Marks corrections with interactive HTML spans

- **Domain Awareness:** Maintains specialized lexicons for academic and technical terminology
- **Output:** Returns enriched text with correction markers for human verification

This function exemplifies how "comments specify the content of a single

Mathematical Expression Handling:

`FormulaCSP.processMathExpressions()`

Context-Aware Formula Processing

This function implements a critical component of the conceptual space framework specifically designed for handling mathematical expressions, demonstrating how "the process in which conceptual spaces are formed" can be applied to technical content.

- **Pattern Recognition:** Identifies both inline ($\$...\$$) and display ($\\\$...\\\$$) math expressions
- **State Management:** Tracks formula context to maintain mathematical integrity
- **Storage Mechanism:**
 - Generates unique keys for each formula (FO####)
 - Stores formulas in the conceptual space repository
 - Creates transclusions for document integration
- **Output:** Replaces raw formulas with conceptual space references

Document Structure: `ItemizeCSP.processParagraph()` & `FootnoteCSP.processParagraph()`

Hierarchical Document Element Processing

These complementary functions implement the processing of structural document elements within the conceptual space framework, demonstrating how "comments are used to remind yourself and to inform others about the details of the code" can be extended to document structure.

- **Itemize Processing:**

- Detects bullet points and numbered lists
- Creates hierarchical item keys (`bibkey.itemize#.item#`)
- Generates dynamic list content via TiddlyWiki filters

- **Footnote Processing:**

- Identifies footnote markers and definitions
- Creates structured footnote references
- Stores footnote content in the conceptual space

- **Common Pattern:** Both convert structural elements to transclusions with conceptual space references

Semantic Enhancement:

CleaningCSP.wordClassification(word)

Context-Aware Word Processing Function

This function implements the semantic enhancement component of the conceptual space framework, demonstrating how "the process in which conceptual spaces are formed" operates at the lexical level.

- **Validation:** Checks word validity against multiple dictionaries (British, American, Academic)
- **Correction Strategy:**
 - Preserves technical terms and proper names
 - Generates suggestions for misspelled words
 - Maintains domain-specific vocabulary
- **Output:** Returns either the original word or its most probable correction
- **Integration:** Works within the larger processing pipeline to maintain document coherence